Chhatrapati Shahu Ji Maharaj University

Project Synopsis

on

AlgoLenz: VisuAlgo-Inspired Interactive Data Structures, Pathfinding & Sorting Visualization Platform

Submitted in Partial Fulfillment of the Requirements for the

Degree of

Bachelors of Computer Application

By

Naitik, 24116002322

Kashish Gupta, 24116002274

Naitik Mishra, 24116002324

Khushboo Rajpoot, 24116002277

Under the Supervision of

Dr. Amit Yadav

Professor

PSIT College of Higher Education

Kanpur-Agra-Delhi National Highway – 2, Bhauti, Kanpur

(2026-2027)

ABSTRACT

Modeled after world-class educational platforms such as VisuAlgo (developed at the National University of Singapore), **AlgoLenz** is an advanced interactive web-based visualization and learning platform designed to transform complex Data Structures and Algorithms (DSA) into intuitive, step-by-step visual experiences. Computer science algorithms—ranging from graph pathfinding to array sorting—often present steep conceptual hurdles when taught solely through static textbooks or code listings.

AlgoLenz bridges this gap by providing real-time, fine-grained animation controls, synchronized pseudocode execution highlighting, live variable state tracking, and interactive canvas environments. The platform supports graph traversals (BFS, DFS), shortest path algorithms (Dijkstra, A*, Bellman-Ford), sorting algorithms (Bubble, Selection, Insertion, Merge, Quick, Heap), multi-stop waypoint path planning (Traveling Salesperson Problem), and real-world geographic map routing using Leaflet.js. Built with React, TypeScript, Vite, Tailwind CSS, and HTML5 Canvas, AlgoLenz also features side-by-side algorithm benchmarking and interactive self-assessment quiz engines to empower computer science students and educators worldwide.

CONTENTS

	Page no.
1. Introduction	3
1.1 Objective	3
1.2 Scope	3
2. System Requirement	4
2.1 Software Requirement	4
2.2 Hardware Requirement	4
3. Modules	5
3.1 Modules Name	5
3.2 Modules Description	5
4. Application Architecture	6
5. Technology Used	7
6. References	8

1. INTRODUCTION

1.1 Objective

Inspired by VisuAlgo, the primary objective of AlgoLenz is to deliver a comprehensive, accessible, and feature-rich interactive algorithm visualizer and educational platform. The core objectives include:

- **VisuAlgo-Style Synchronized Pseudocode:** Provide line-by-line code highlighting during live algorithm execution to clearly connect abstract code logic with visual graph/array states.
- **Interactive Control & Data Input:** Allow users to pause, step forward/backward, adjust speed, draw custom walls, set terrain weights, generate random inputs, and edit graph node configurations.
- **Multi-Paradigm Algorithm Suite:** Cover graph traversals (BFS, DFS), weighted shortest path search (Dijkstra, A*, Bellman-Ford), and comparison/divide-and-conquer sorting (Bubble, Selection, Insertion, Merge, Quick, Heap).
- **Concurrent Multi-Algorithm Comparison:** Run multiple algorithms side-by-side on identical input data to benchmark execution runtime (ms), path cost, visited nodes, and array swaps.
- **Spatial & Geographic Routing:** Solve multi-stop waypoint path optimization (Traveling Salesperson Problem) and overlay graph algorithms onto real-world Leaflet geographic maps.
- **Self-Assessment & Educational Tools:** Provide integrated e-lecture mode summaries and automated quizzes to evaluate algorithm comprehension.

1.2 Scope

The scope of AlgoLenz spans full frontend engineering, algorithm engine design, dynamic canvas rendering, state tracking, and pedagogical tool creation:

- **Graph & Pathfinding Engine:** Dijkstra, A* Search, BFS, DFS, and Bellman-Ford algorithms with 2D grid and weighted edge support.
- **Sorting Engine:** Bubble, Selection, Insertion, Merge, Quick, and Heap Sort with real-time bar height rendering and swap counters.
- **VisuAlgo Inspector & Pseudocode Panel:** Real-time call stack tracking, variable value inspector, and active line-by-line pseudocode highlighting.
- **Map & Waypoint Module:** Traveling Salesperson Problem (TSP) solver and Leaflet map geographic route visualization.
- **Analytics & Quiz Engine:** Benchmark comparison cards and self-paced assessment quizzes.

2. SYSTEM REQUIREMENTS

2.1 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Runtime Environment:** Node.js (v18.x or higher), npm (v9.x or higher)
- **Frontend Framework & Language:** React 18, React-DOM, TypeScript 5.3
- **Build Tooling & Bundler:** Vite 5, PostCSS, Autoprefixer
- **Styling & UI System:** Tailwind CSS 3.4, Lucide React (Icons)
- **Geospatial & Mapping Libraries:** Leaflet 1.9, React-Leaflet 4.2
- **Supported Web Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (HTML5 Canvas & ES6 compliant)

2.2 Hardware Resource Requirements

- **Processor:** Intel Core i3 / i5 or AMD Ryzen 3 / 5 equivalent (2.0 GHz Dual-Core minimum)
- **RAM:** Minimum 4 GB (8 GB recommended for concurrent multi-canvas side-by-side rendering)
- **Storage:** 500 MB available disk space for Node modules and build assets
- **Display Resolution:** Minimum 1366x768 (1080p Full HD recommended)

3. MODULES

3.1 Modules Name

1. VisuAlgo-Style Interactive Grid & Graph Visualization Engine
2. Sorting Algorithm & Array Animation Module
3. Synchronized Pseudocode & State Inspector Module
4. Side-by-Side Multi-Algorithm Comparison & Benchmarking Module
5. Multi-Stop Waypoint & TSP Optimization Engine
6. Real-World Geographic City Map Routing Engine (Leaflet)
7. Interactive E-Lecture & Self-Assessment Quiz Module

3.2 Modules Description

- **Interactive Grid & Graph Visualization Engine:** Renders a 2D grid canvas supporting start/target node positioning, dynamic wall obstacle drawing, terrain weight painting, and step-by-step animation of search frontiers and shortest paths.
- **Sorting Algorithm & Array Animation Module:** Displays array sorting algorithms with dynamic bar charts, highlighting active comparisons, pivot selections, array element swaps, and sorted sub-arrays in real time.
- **Synchronized Pseudocode & State Inspector Module:** Displays pseudocode for the active algorithm, dynamically highlighting the currently executing line alongside real-time variable values, queue/stack contents, and step explanations (VisuAlgo standard).
- **Side-by-Side Multi-Algorithm Comparison Module:** Concurrently runs two algorithms on identical input setups to benchmark runtime speed (ms), visited node count, path length, array swaps, and theoretical time/space complexity.
- **Multi-Stop Waypoint & TSP Engine:** Computes the optimal visiting path across multiple target waypoints using dynamic programming and nearest-neighbor heuristics for the Traveling Salesperson Problem.

4. APPLICATION ARCHITECTURE

1. **User Interface & Control Panel Layer:** Built using React functional components, Lucide icons, and Tailwind CSS. Provides top navigation, mode switcher, playback control bar (play/pause, step, speed slider, reset), and input data generators.
2. **State Management & Context Pipeline:** Managed via centralized `VisualizerContext`. Tracks global application state including grid structure, obstacle configurations, selected algorithms, speed factors, step animation queues, and performance metrics.
3. **Algorithmic Engine & Pseudocode Synchronizer:** Modular TypeScript engine containing pathfinding (`dijkstra.ts`, `aStar.ts`, `bfs.ts`, `dfs.ts`, `bellmanFord.ts`) and sorting (`sorting.ts`). Generates step snapshots bound to specific pseudocode line index references.
4. **Dual Visualization & Graphics Engine:** High-frequency HTML5 Canvas rendering for 2D grids and sorting bar charts, coupled with Leaflet/SVG vector layers for real-world geographic map graphs.
5. **Analytics, Benchmarking & Quiz Engine:** Measures execution time (ms), total path distance, visited nodes, array operations, asymptotic complexity metrics ($O(V+E)$, $O(n \log n)$), and processes interactive quiz assessments.

5. TECHNOLOGY USED

5.1 Frontend Technologies:

- **React 18:** Functional UI library utilizing hooks for reactive layout and component composition.
- **TypeScript 5.3:** Static type checking ensuring reliable algorithm implementation and step state definitions.
- **HTML5 Canvas API & SVG:** High-performance graphic rendering for grid animations and vector overlays.

5.2 Styling & Design System:

- **Tailwind CSS 3.4:** Utility-first CSS framework enabling modern responsive design and dark/beige themes.
- **Lucide React:** Crisp icon set for playback controls, status badges, and navigation items.

5.3 Mapping & Geospatial Libraries:

- **Leaflet 1.9 & React-Leaflet:** Open-source JavaScript mapping library for real-world geographic node routing.

5.4 Build System & Tooling:

- **Vite 5:** Lightning-fast frontend build tool with Hot Module Replacement (HMR) and optimized rollup production bundles.
- **Node.js & npm:** Package management and execution environment.

6. REFERENCES

1. **VisuAlgo Platform & Conceptual Foundation:** -
   Halim, S., & Halim, F. (2013). *Competitive Programming 3: The New Lower Bound of Programming Contests* (VisuAlgo Authors, National University of Singapore). https://visualgo.net/

2. **Standard Algorithm & Data Structure Literature (CLRS):** -
   Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

3. **Pathfinding & Graph Theory Papers:** -

   3.1. Dijkstra's Algorithm: Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.

   3.2. A* Search Heuristic: Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.

4. **Framework & Library Documentation:** -

   4.1. React.js Documentation: https://react.dev/

   4.2. TypeScript Documentation: https://www.typescriptlang.org/

   4.3. Leaflet Interactive Maps API: https://leafletjs.com/

   4.4. Vite Build Tooling: https://vitejs.dev/